

# PES UNIVERSITY, BANGALORE

## Department of Computer Science and Engineering

Software Engineering (UE24CS341A) — Jackfruit Mini-Project

### DELIVERABLE 2: REQUIREMENTS PACKAGE

*Software Requirements Specification (SRS), Validation Specification & Project Plan*

|                    |                                                                                                                                                        |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project Title      | PassPulse – Smart Event Discovery, High-Concurrency Ticketing & Anti-Fraud Check-in Platform                                                           |
| Group & Project ID | Team 2   Project ID: PP-2026-T02                                                                                                                       |
| Team Members       | 1. Sanchita Sunil (PES1UG24CS419) – Team Lead<br>2. Saatvik Gupta (PES1UG24CS395)<br>3. Reet V Porwal (PES1UG24CS369)<br>4. Sambhram M (PES1UG24CS409) |
| Target Technology  | React.js, Node.js (Express), PostgreSQL, Redis, Docker, Jenkins, SonarQube                                                                             |

#### Revision History

| Version | Date        | Author(s)                         | Reason For Changes                              |
|---------|-------------|-----------------------------------|-------------------------------------------------|
| 1.0     | 03-Sep-2026 | Team 2                            | Initial Project Synopsis submission             |
| 2.0     | 07-Sep-2026 | Sanchita, Saatvik, Reet, Sambhram | Deliverable 2: Full SRS, Validation Plan & SPMP |

#### Table of Contents

1. Introduction (Purpose, Intended Audience, Product Scope, References)
2. Overall Description (Product Perspective, Functions, User Classes, Operating Environment, Design Constraints, Assumptions)
3. External Interface Requirements (User Interfaces, Software Interfaces, Communications Interfaces, Hardware Interfaces)
4. Analysis Models (Use Case Model, Conceptual Domain Model)
5. Detailed System Features & Functional Requirements
6. Non-Functional Requirements (Performance, Safety, Security, Quality Attributes, Business Rules)
7. Other Requirements (Database, Data Retention, Deployment)
8. Appendix A: Glossary & Acronyms
9. Appendix B: Field Layouts & Data Dictionary
10. Appendix C: Requirement Traceability Matrix (RTM)
11. Validation Specification (Initial Planning Stage)
12. Software Project Plan (SPMP & WBS)

## 1. Introduction

### 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to specify the complete requirements for **PassPulse** (Version 2.0). It covers the functional behavior, external interfaces, performance metrics, and operational constraints for the system. This document forms the baseline for design and implementation during the semester under course UE24CS341A.

### 1.2 Intended Audience

This document is intended for course instructors, faculty evaluators, the student development team (Team 2), and peer reviewers to understand the project scope, technical requirements, and test planning.

### 1.3 Product Scope

PassPulse is a web platform for managing event discovery, multi-tier ticket booking, dynamic QR check-in, and attendance tracking.

| In Scope                                                                                                                                       | Out of Scope                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• User authentication with Role-Based Access Control (Attendee, Organizer, Gate Staff, Admin).</li></ul> | <ul style="list-style-type: none"><li>• Real commercial banking payment gateway integration (simulated mock checkout will be used).</li></ul> |
| <ul style="list-style-type: none"><li>• Event creation, publishing, editing, scheduling, and multi-tier pricing setup.</li></ul>               | <ul style="list-style-type: none"><li>• Live video streaming or virtual stage broadcasting.</li></ul>                                         |
| <ul style="list-style-type: none"><li>• Concurrency-controlled ticket booking with 10-minute temporary TTL reservation holds.</li></ul>        | <ul style="list-style-type: none"><li>• AI facial recognition check-in systems.</li></ul>                                                     |
| <ul style="list-style-type: none"><li>• Dynamic, cryptographic HMAC-signed QR ticket generation to prevent screenshot sharing.</li></ul>       | <ul style="list-style-type: none"><li>• Complex multi-campus hardware RFID / turnstile integration.</li></ul>                                 |
| <ul style="list-style-type: none"><li>• In-app camera scanner for gate check-in in under 200 ms.</li></ul>                                     | <ul style="list-style-type: none"><li>• Native mobile apps on App Store/Play Store (mobile responsive web used).</li></ul>                    |
| <ul style="list-style-type: none"><li>• Automated FIFO waitlist reallocation on cancellations with a 15-minute claim timer.</li></ul>          | <ul style="list-style-type: none"><li>• Full accounting / ERP integration.</li></ul>                                                          |
| <ul style="list-style-type: none"><li>• Real-time attendance dashboards and downloadable summary reports (CSV/PDF).</li></ul>                  | <ul style="list-style-type: none"><li>• Venue indoor GPS navigation maps.</li></ul>                                                           |

### 1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- OWASP Top 10 Web Application Security Risks (2021).
- RFC 7519: JSON Web Token (JWT) Standard.
- PES University Department of CSE: Jackfruit Mini-Project Guidelines (UE24CS341A).

## 2. Overall Description

### 2.1 Product Perspective

PassPulse is an independent web application. The frontend is built as a React.js single-page application, and the backend is implemented with Node.js and Express.js REST APIs. PostgreSQL is used for relational data storage, and Redis handles in-memory locks for temporary ticket holds and waitlist queues.

## 2.2 Product Functions

- **Account Management:** User registration, login, role-based authorization, and profile handling.
- **Event Catalog:** Search and filtering of events by category, date, price, and venue.
- **Ticket Management:** Quota creation for General, Early Bird, and VIP categories.
- **Atomic Ticket Booking:** 10-minute temporary inventory holds to prevent overselling.
- **Dynamic QR Pass Issuance:** Cryptographically signed digital passes generated for bookings.
- **Gate Scanner:** Camera-based QR verification and duplicate entry prevention.
- **Waitlist System:** FIFO re-allocation to waitlisted attendees upon cancellations.
- **Attendance Reports:** Live check-in velocity and exportable attendance reports.

## 2.3 User Classes and Characteristics

| User Class  | Privileges                                                                               | Characteristics                                                                                  |
|-------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Attendee    | Browse events, reserve tickets, view QR passes, cancel bookings, join waitlists.         | General public/students; non-technical; creates burst traffic during ticket release windows.     |
| Organizer   | Create and edit events, configure ticket tiers, monitor sales, export attendance data.   | Club leads and event managers; requires clear form workflows and analytical reports.             |
| Gate Staff  | Access in-app camera scanner, validate QR tickets, mark attendance.                      | Volunteers and gate personnel; requires fast scan response (<200ms) with clear pass/fail status. |
| Super-Admin | Approve or flag public event listings, manage user accounts, review platform audit logs. | Platform administrators; requires elevated permissions and governance oversight.                 |

## 2.4 Operating Environment

- **Server Environment:** Linux Ubuntu 22.04 LTS / Docker containers with Node.js v20.x.
- **Database & Cache:** PostgreSQL 16.x and Redis Server 7.x.
- **Client Browsers:** Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari.
- **Tooling & CI/CD:** Docker Engine, Jenkins, SonarQube, Git / GitHub.

## 2.5 Design and Implementation Constraints

- **Concurrency Safety:** Ticket inventory decrements must execute inside database transactions protected by Redis locks to prevent overselling.
- **Stateless Authentication:** All API requests must use cryptographically signed JWT bearer tokens.
- **Gate Latency:** QR validation and check-in confirmation must execute in under 200 ms.
- **Code Standards:** ESLint compliance, 80% test coverage in Jest, and zero critical vulnerabilities in SonarQube.

## 2.6 Assumptions and Dependencies

- Gate staff devices have functional cameras and an active network connection (minimum 1 Mbps).
- Attendees access the platform using standard modern desktop or mobile web browsers.
- Application, database, and Redis servers have synchronized system clocks.

### 3. External Interface Requirements

#### 3.1 User Interfaces

- **Discovery & Catalog View:** Grid of event cards with search bar, category filters, date filters, and price ranges.
- **Event Creation Wizard:** Multi-step form for event title, venue, date/time, description, banner upload, and ticket tiers.
- **Checkout & Hold Screen:** 10-minute countdown timer for temporary ticket holds, order summary, and confirmation button.
- **Dynamic QR Ticket Pass:** Mobile pass view showing attendee name, event details, tier label, and dynamic QR code.
- **Gate Scanner View:** Camera viewfinder with visual indicators (Green = Valid, Red = Already Checked-In / Invalid).
- **Organizer Telemetry Dashboard:** Check-in progress bar, check-in velocity chart, and CSV/PDF export buttons.

#### 3.2 Software Interfaces

- **Database Interface:** PostgreSQL connection pool accessed via Sequelize / Prisma ORM with parameterized queries.
- **Cache Interface:** Redis client connection for key expiration events, distributed locking, and waitlist queues.
- **Email Service:** Nodemailer integration over SMTP for booking receipts and waitlist claim invitations.

#### 3.3 Communications Interfaces

- **API Protocol:** HTTPS REST APIs using JSON payloads.
- **Authorization Header:** Standard Bearer JWT header transmitted over TLS 1.3.
- **Live Telemetry:** Server-Sent Events (SSE) / WebSocket connections for live gate check-in counts.

#### 3.4 Hardware Interfaces

- **Camera Hardware:** Standard HTML5 MediaDevices API accessing built-in device cameras for live QR scanning.

## 4. Analysis Models

### 4.1 Use Case Matrix

| ID    | Use Case Name              | Primary Actor     | Preconditions                         | Outcome                                   |
|-------|----------------------------|-------------------|---------------------------------------|-------------------------------------------|
| UC-01 | Register & Login           | All Users         | Valid credentials format              | Authenticated session & JWT token issued  |
| UC-02 | Search & Filter Events     | Attendee          | Application accessible                | Filtered event list displayed             |
| UC-03 | Create & Publish Event     | Organizer         | Organizer authenticated               | Event created and marked active           |
| UC-04 | Hold Ticket (10-Min TTL)   | Attendee          | Attendee logged in; quota available   | Selected tickets held; timer started      |
| UC-05 | Confirm Booking & Get Pass | Attendee          | Valid hold active; checkout completed | Booking saved; HMAC QR pass generated     |
| UC-06 | Join Waitlist Queue        | Attendee          | Selected ticket tier is sold out      | User placed in FIFO waitlist              |
| UC-07 | Scan QR at Gate            | Gate Staff        | Staff authenticated; camera active    | Pass verified; attendee marked checked-in |
| UC-08 | Export Attendance Report   | Organizer / Admin | Event active or completed             | Attendance summary generated (CSV/PDF)    |

### 4.2 Conceptual Domain Model

- **USER (1) — (M) EVENT:** Organizers create and manage events.
- **EVENT (1) — (M) TICKET\_TIER:** Each event defines one or more ticket tiers with specific quotas and prices.
- **TICKET\_TIER (1) — (M) BOOKING:** Attendees reserve ticket slots within available tier capacities.
- **BOOKING (1) — (M) QR\_PASS:** A confirmed booking produces digital QR passes per attendee.
- **QR\_PASS (1) — (1) CHECKIN\_LOG:** Each QR pass produces exactly one verified check-in record at the gate.
- **TICKET\_TIER (1) — (M) WAITLIST\_ENTRY:** When tier capacity is 0, users join the tier waitlist queue.

## 5. Detailed System Features & Functional Requirements

### 5.1 User Authentication & Role-Based Access Control (RBAC)

**Description:** Provides registration, login, role enforcement, and token-based session handling.  
(Priority: High)

| Req ID | Requirement Description                                                                                                     | Priority | Verification       |
|--------|-----------------------------------------------------------------------------------------------------------------------------|----------|--------------------|
| FR-001 | The system shall allow users to register with a unique email address, password, full name, and role selection.              | High     | Functional Test    |
| FR-002 | The system shall hash passwords using bcrypt (\$\text{salt rounds}\ge 10\$) prior to persistence.                           | High     | Security Test      |
| FR-003 | The system shall authenticate requests using JWT bearer tokens and reject unauthorized role access with HTTP 403 Forbidden. | High     | Authorization Test |

### 5.2 Event Discovery & Lifecycle Management

**Description:** Allows attendees to search events and organizers to manage event publication states.  
(Priority: High)

| Req ID | Requirement Description                                                                                                      | Priority | Verification          |
|--------|------------------------------------------------------------------------------------------------------------------------------|----------|-----------------------|
| FR-004 | The system shall allow users to filter events by keyword, category, date range, venue, and ticket price.                     | High     | Functional Test       |
| FR-005 | The system shall allow organizers to create, update, schedule, and cancel events with title, venue, capacity, and date/time. | High     | Functional Test       |
| FR-006 | The system shall maintain event states: DRAFT, PENDING_APPROVAL, PUBLISHED, LIVE, COMPLETED, and CANCELLED.                  | Medium   | State Transition Test |

### 5.3 Ticket Booking & Concurrency Hold Engine

**Description:** Handles seat allocation and prevents overbooking using temporary atomic holds.  
(Priority: High)

| Req ID | Requirement Description                                                                                                           | Priority | Verification      |
|--------|-----------------------------------------------------------------------------------------------------------------------------------|----------|-------------------|
| FR-007 | The system shall allow organizers to configure ticket tiers (e.g., General, Early Bird, VIP) with independent quotas and pricing. | High     | Functional Test   |
| FR-008 | The system shall temporarily hold selected tickets for exactly 10 minutes using Redis locks during checkout.                      | High     | Concurrency Test  |
| FR-009 | The system shall automatically release held inventory back to available stock if checkout is not completed within 10 minutes.     | High     | Negative/TTL Test |
| FR-010 | The system shall commit confirmed bookings inside a PostgreSQL ACID transaction to guarantee zero overselling.                    | High     | Integration Test  |

### 5.4 Dynamic Cryptographic QR Pass Generation

**Description:** Issues digital passes for confirmed bookings to prevent screenshot sharing. (Priority: High)

| Req ID | Requirement Description                                                                                                           | Priority | Verification |
|--------|-----------------------------------------------------------------------------------------------------------------------------------|----------|--------------|
| FR-011 | The system shall generate a unique cryptographic digital QR code for each confirmed ticket, signed using HMAC-SHA256.             | High     | Unit Test    |
| FR-012 | The system shall provide attendees with an interactive digital pass displaying attendee details, ticket tier, and active QR view. | High     | UI Test      |

### 5.5 Gate Check-in Scanner & Validation

**Description:** Enables gate staff to validate attendee passes rapidly and prevent duplicate entries.  
(Priority: High)

| Req ID | Requirement Description                                                                                                                | Priority | Verification     |
|--------|----------------------------------------------------------------------------------------------------------------------------------------|----------|------------------|
| FR-013 | The system shall allow authorized Gate Staff to scan QR passes using device cameras and decode verification payloads in real time.     | High     | Functional Test  |
| FR-014 | The system shall mark validated passes as CHECKED_IN, record entry timestamp, and reject duplicate scans with a DUPLICATE_ENTRY alert. | High     | Integration Test |

### 5.6 Automated Waitlist Engine

**Description:** Manages queueing when tiers sell out and re-allocates slots upon cancellations.  
(Priority: Medium)

| Req ID | Requirement Description                                                                                                              | Priority | Verification     |
|--------|--------------------------------------------------------------------------------------------------------------------------------------|----------|------------------|
| FR-015 | The system shall allow attendees to join a FIFO waitlist when a ticket tier reaches zero available quota.                            | Medium   | Functional Test  |
| FR-016 | Upon ticket cancellation, the system shall notify the next waitlisted user via email and lock the slot for a 15-minute claim window. | Medium   | Integration Test |

### 5.7 Telemetry, Reports & Administration

**Description:** Provides live check-in graphs, summary exports, and administrator governance.  
(Priority: Medium)

| Req ID | Requirement Description                                                                                                                     | Priority | Verification       |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------|----------|--------------------|
| FR-017 | The system shall display real-time gate entry velocity (scans per minute) and attendance percentages on the organizer dashboard.            | Medium   | System Test        |
| FR-018 | The system shall generate downloadable attendance and revenue summary reports in CSV and PDF formats.                                       | Medium   | Functional Test    |
| FR-019 | The system shall provide Administrators with tools to review, approve, flag, or remove public events and deactivate non-compliant accounts. | High     | Authorization Test |

## 6. Non-Functional Requirements

### 6.1 Performance Requirements

| ID      | Performance Target                                                                                            | Verification Method      |
|---------|---------------------------------------------------------------------------------------------------------------|--------------------------|
| NFR-001 | 95% of event search and catalog queries shall respond within 150 ms under a load of 500 concurrent users.     | Load Test (Postman / k6) |
| NFR-002 | QR pass cryptographic validation and check-in database commit shall complete in under 200 ms per scan.        | Latency Benchmark Test   |
| NFR-003 | The booking engine shall process 200 concurrent checkout hold requests per second without data inconsistency. | Concurrency Stress Test  |

### 6.2 Safety Requirements

- The system shall enforce maximum venue capacity ceilings to prevent overbooking physical halls.
- In case of server restart or database failover, all confirmed booking and check-in records shall be fully recoverable from database write-ahead logs.

### 6.3 Security Requirements

| ID      | Security Requirement                                                                                          | Verification Method  |
|---------|---------------------------------------------------------------------------------------------------------------|----------------------|
| SEC-001 | User passwords shall be hashed using bcrypt ( $\geq 10$ ) and never stored in plaintext.                      | Code / Config Review |
| SEC-002 | Role-Based Access Control (RBAC) shall prevent attendees from accessing organizer or admin routes.            | Authorization Tests  |
| SEC-003 | User input shall be validated and sanitized before database queries to prevent SQLi and XSS vulnerabilities.  | Security / DAST Scan |
| SEC-004 | All external network communication between client and server shall be encrypted using TLS 1.3.                | Network Inspection   |
| SEC-005 | Security-relevant events (failed logins, unauthorized scans, role elevation) shall be logged in audit tables. | Audit Log Review     |

### 6.4 Software Quality Attributes

- **Availability:** The system shall maintain  $\geq 99.5\%$  operational uptime during active event registration and check-in windows.
- **Maintainability:** Layered architecture (Controllers, Services, Repositories) with automated linting rules.
- **Testability:** Unit tests must achieve  $\geq 80\%$  statement and branch coverage via Jest / Supertest test suites.
- **Usability:** An attendee shall be able to discover an event and complete ticket reservation in no more than 4 primary screens.



## 6.5 Business Rules

| Rule ID | Business Rule Definition                                                                                                                     |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------|
| BR-001  | A single user cannot reserve more than 5 tickets per booking transaction for any single event.                                               |
| BR-002  | A ticket hold expires automatically after 600 seconds (10 minutes) if checkout is not confirmed.                                             |
| BR-003  | A QR ticket pass can be marked as CHECKED_IN only once; subsequent scans must be rejected immediately.                                       |
| BR-004  | Waitlist slot claim invitations expire strictly after 15 minutes, after which the slot is automatically offered to the next waitlisted user. |
| BR-005  | An event cannot be published unless it has at least one active ticket tier with capacity greater than zero.                                  |
| BR-006  | Organizers cannot delete or modify event dates once confirmed bookings have been issued without issuing an automated cancellation notice.    |

## 7. Other Requirements

- **Database Persistence:** All core entities (users, events, bookings, tickets, check-in logs) must persist in PostgreSQL with enforced foreign key integrity.
- **Data Retention:** Event attendance records and logs shall be retained for 1 year for auditing and reporting purposes.
- **Deployment:** The full stack (React SPA, Express API, PostgreSQL, Redis) must deploy via standard `docker-compose up -d`.

Appendix A: Glossary & Acronyms

| Term             | Definition                                                                                                             |
|------------------|------------------------------------------------------------------------------------------------------------------------|
| RBAC             | Role-Based Access Control: Restricts system feature access based on assigned user roles.                               |
| JWT              | JSON Web Token: Stateless, cryptographically signed token used for API authentication.                                 |
| TTL              | Time-To-Live: Automatic key expiration timer in Redis (used for 10-minute ticket holds).                               |
| HMAC             | Hash-based Message Authentication Code: Cryptographic signature algorithm (HMAC-SHA256) used for QR pass verification. |
| FIFO             | First-In, First-Out: Queueing rule where the earliest waitlisted attendee receives the first available slot.           |
| Ingress Velocity | The number of attendees validated and checked in per minute at venue entry gates.                                      |

Appendix B: Field Layouts & Data Dictionary

Table 1: User Accounts (`users`)

| Field Name      | Length | Data Type | Description                               | Mandatory |
|-----------------|--------|-----------|-------------------------------------------|-----------|
| `user_id`       | 36     | UUID      | Primary key unique identifier             | Yes (PK)  |
| `full_name`     | 100    | VARCHAR   | User full legal name                      | Yes       |
| `email`         | 150    | VARCHAR   | Unique user email address                 | Yes (UQ)  |
| `password_hash` | 255    | VARCHAR   | Bcrypt hashed password string             | Yes       |
| `role`          | 20     | ENUM      | 'ATTENDEE', 'ORGANIZER', 'STAFF', 'ADMIN' | Yes       |
| `created_at`    | 8      | TIMESTAMP | Account creation timestamp (UTC)          | Yes       |

Table 2: Event Details (`events`)

| Field Name     | Length | Data Type | Description                                  | Mandatory |
|----------------|--------|-----------|----------------------------------------------|-----------|
| `event_id`     | 36     | UUID      | Primary key unique identifier                | Yes (PK)  |
| `organizer_id` | 36     | UUID      | Foreign key referencing `users.user_id`      | Yes (FK)  |
| `title`        | 200    | VARCHAR   | Public event title                           | Yes       |
| `category`     | 50     | VARCHAR   | 'Tech', 'Cultural', 'Conference', 'Workshop' | Yes       |
| `venue`        | 200    | VARCHAR   | Venue auditorium or hall name                | Yes       |
| `event_date`   | 8      | TIMESTAMP | Scheduled event start datetime               | Yes       |
| `status`       | 20     | ENUM      | 'DRAFT', 'PUBLISHED', 'LIVE', 'COMPLETED'    | Yes       |

Table 3: Ticket Tiers (`ticket\_tiers`)

| Field Name        | Length | Data Type | Description                               | Mandatory |
|-------------------|--------|-----------|-------------------------------------------|-----------|
| `tier_id`         | 36     | UUID      | Primary key unique identifier             | Yes (PK)  |
| `event_id`        | 36     | UUID      | Foreign key referencing `events.event_id` | Yes (FK)  |
| `tier_name`       | 50     | VARCHAR   | 'Early Bird', 'General', 'VIP'            | Yes       |
| `price`           | 10,2   | DECIMAL   | Ticket price amount                       | Yes       |
| `total_quota`     | 6      | INTEGER   | Total capacity allocated for tier         | Yes       |
| `available_quota` | 6      | INTEGER   | Remaining unreserved stock                | Yes       |

Sample Report Layout: Post-Event Attendance Audit Report

| Event Details                                                                                                                                               | Registration Summary                                                                                                                                    | Gate Attendance Metrics                                                                                                                                     | Revenue Breakdown                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>Event ID: `EV-9042`</li><li>Title: TechSummit 2026</li><li>Venue: Main Auditorium</li><li>Date: 15-Oct-2026</li></ul> | <ul style="list-style-type: none"><li>Total Capacity: 500</li><li>Tickets Booked: 495</li><li>Waitlist Requests: 42</li><li>Cancellations: 12</li></ul> | <ul style="list-style-type: none"><li>Checked-In: 460</li><li>Absent / No-Show: 35</li><li>Turnout Rate: 92.9%</li><li>Peak Scan Velocity: 34/min</li></ul> | <ul style="list-style-type: none"><li>Early Bird: 150 @ ₹199</li><li>General: 300 @ ₹399</li><li>VIP: 45 @ ₹799</li><li>Total: ₹1,85,505</li></ul> |

## Appendix C: Requirement Traceability Matrix (RTM)

| Sl. | Req ID | Brief Description                    | Architecture Module  | Code File Reference                     | Unit Test ID | System Test ID |
|-----|--------|--------------------------------------|----------------------|-----------------------------------------|--------------|----------------|
| 1   | FR-001 | User Registration & Email Uniqueness | Auth Service         | `src/controllers/auth.controller.js`    | UT-AUTH-01   | ST-REG-01      |
| 2   | FR-002 | Bcrypt Password Hashing              | Security Middleware  | `src/utils/password.util.js`            | UT-AUTH-02   | ST-SEC-01      |
| 3   | FR-003 | JWT Token Verification & RBAC        | Auth Middleware      | `src/middleware/rbac.middleware.js`     | UT-AUTH-03   | ST-SEC-02      |
| 4   | FR-004 | Event Catalog Search & Filtering     | Event Module         | `src/services/event.service.js`         | UT-EVT-01    | ST-EVT-01      |
| 5   | FR-005 | Event Creation & Lifecycle State     | Event Module         | `src/controllers/event.controller.js`   | UT-EVT-02    | ST-EVT-02      |
| 6   | FR-007 | Multi-Tier Ticket Configuration      | Ticket Service       | `src/models/ticketTier.model.js`        | UT-TCK-01    | ST-TCK-01      |
| 7   | FR-008 | Atomic 10-Min TTL Ticket Hold        | Booking Engine       | `src/services/booking.service.js`       | UT-BKG-01    | ST-CONC-01     |
| 8   | FR-010 | ACID Transaction Booking Commit      | Booking Engine       | `src/repositories/booking.repo.js`      | UT-BKG-02    | ST-BKG-01      |
| 9   | FR-011 | Dynamic HMAC-Signed QR Pass          | QR Generator Service | `src/services/qr.service.js`            | UT-QR-01     | ST-QR-01       |
| 10  | FR-014 | Gate Check-in & Duplicate Rejection  | Scanner Service      | `src/controllers/scanner.controller.js` | UT-SCN-01    | ST-SCN-01      |
| 11  | FR-016 | FIFO Waitlist 15-Min Reallocation    | Waitlist Worker      | `src/workers/waitlist.worker.js`        | UT-WTL-01    | ST-WTL-01      |
| 12  | FR-017 | Live Attendance Telemetry Stream     | Analytics Service    | `src/services/analytics.service.js`     | UT-ANL-01    | ST-ANL-01      |

### Validation Specification (Initial Planning Stage)

This section defines the planned testing strategies, verification mechanisms, and security checks to be executed during subsequent implementation sprints.

## 1. Testing Methodologies

- **Black-Box Functional Testing:** Verification of user workflows (registration, event creation, ticket booking, waitlist claiming) using boundary value analysis and equivalence class partitioning.
- **Security & Penetration Testing:**
  - Authorization Bypass: Attempting to invoke `/api/admin/\*` and `/api/organizer/\*` endpoints using attendee JWT tokens to verify HTTP 403 response.
  - QR Replay Attacks: Attempting to scan the same QR payload twice to verify that the second scan triggers a DUPLICATE\_ENTRY exception.
  - Injection Testing: Submitting malformed JSON payloads with SQL statements and script tags to verify input sanitization.
- **Concurrency & Load Testing:** Simulating 200 concurrent HTTP POST requests to reserve the last remaining ticket in a tier using Postman / k6 runner to verify zero overselling.
- **Automated Unit & Integration Testing:** Unit test suites authored in Jest and Supertest targeting a minimum of 80% line and branch coverage integrated into the CI/CD pipeline.

## Software Project Plan (SPMP & WBS)

### 1. Agile Lifecycle Model & Justification

The project follows the **Agile Scrum Methodology** divided into six 2-week sprints. Agile was selected because the system requires progressive refinement of complex features (concurrency hold, cryptographic QR, waitlist workers), continuous integration via Jenkins, and weekly evaluation milestones.

### 2. Work Breakdown Structure (WBS) & Effort Estimation

| Package | Work Package Description              | Primary Owner     | Effort (Days) | Deliverable Output                  |
|---------|---------------------------------------|-------------------|---------------|-------------------------------------|
| WP1     | Requirements Specification & Baseline | All Members       | 4 Person-Days | Approved SRS v2.0 Document          |
| WP2     | Architecture, UML & DB Modeling       | Sanchita, Saatvik | 4 Person-Days | High-Level Architecture & ER Schema |
| WP3     | User Auth, RBAC & Security Middleware | Saatvik Gupta     | 6 Person-Days | Auth APIs + Security Test Suites    |
| WP4     | Event Catalog & Lifecycle Management  | Sanchita Sunil    | 6 Person-Days | Event Wizard UI + Search Endpoints  |
| WP5     | Concurrency Booking & Waitlist Engine | Reet V Porwal     | 7 Person-Days | Redis Hold Service + ACID Checkout  |
| WP6     | QR Scanner & Attendance Telemetry     | Sambhram M        | 7 Person-Days | Gate Scanner View + Ingress Charts  |
| WP7     | Testing, CI/CD & Static Analysis      | All Members       | 6 Person-Days | Jenkins Pipeline + SonarQube Gate   |
| WP8     | System Integration, Demo & Report     | All Members       | 4 Person-Days | Dockerized Product + Final Report   |

### 3. Sprint Schedule & Tool Stack Mapping

- **Sprint 1 (Weeks 1–2):** SRS Baseline, Jira Backlog Setup, GitHub repo configuration.
- **Sprint 2 (Weeks 2–3):** Authentication API, RBAC Middleware, User Profile Endpoints (Saatvik).

- **Sprint 3 (Weeks 3–4):** Event Creation Wizard, Multi-Tier Pricing Setup, Search Catalog (Sanchita).
- **Sprint 4 (Weeks 5–6):** Concurrency Hold Engine, Atomic Checkout, Automated Waitlist (Reet).
- **Sprint 5 (Weeks 6–7):** Dynamic QR Generator, Camera Gate Scanner, Telemetry Dashboard (Sambhram).
- **Sprint 6 (Weeks 7–8):** Admin Moderation, Dockerization, Jenkins CI Pipeline, Final Project Report (All).